Vysoké učení technické v Brně
Fakulta informačních technologií

Projektová dokumentácia k predmetu ISA
Exportér SNMP Gauge metrik do OpenTelemetry (OTEL)

16. listopadu 2025 Jozef Sládeček (xsladej00)

Obsah

1 Uvedenie do problematiky **3**
- 1.1 Použité technológie a protokoly . . . 3
  - 1.1.1 SNMP (Simple Network Management Protocol) . . . 3
  - 1.1.2 ASN.1 BER kódovanie . . . 3
  - 1.1.3 OpenTelemetry a OTLP/HTTP . . . 3

2 Návrh a implementácia **3**
- 2.1 Architektúra programu . . . 3
- 2.2 SNMP Packet Builder . . . 3
  - 2.2.1 Načítanie OID zo súboru . . . 3
  - 2.2.2 Konverzia OID na ASN.1 BER formát . . . 4
  - 2.2.3 Zostavenie SNMP GetRequest PDU . . . 4
  - 2.2.4 Finálne zostavenie SNMP paketu . . . 4
- 2.3 SNMP Client . . . 4
  - 2.3.1 Inicializácia socketu . . . 4
  - 2.3.2 Odosielanie a príjem paketov . . . 4
- 2.4 SNMP Parser . . . 4
  - 2.4.1 Parsovanie štruktúry paketu . . . 4
  - 2.4.2 Parsovanie OID . . . 5
  - 2.4.3 Pomocné parsovacie funkcie . . . 5
  - 2.4.4 Parsovanie hodnôt . . . 5
- 2.5 OTEL Exporter . . . 5
  - 2.5.1 Filtrovanie numerických hodnôt . . . 5
  - 2.5.2 Zostavenie OTLP JSON . . . 6
  - 2.5.3 HTTP POST pomocou cpp-httplib . . . 6
- 2.6 Hlavný program . . . 6
  - 2.6.1 Parsovanie argumentov . . . 6
  - 2.6.2 Periodické dotazovanie . . . 6
  - 2.6.3 Retry logika . . . 7

3 Zaujímavé časti implementácie **7**
- 3.1 OID kódovanie v ASN.1 BER formáte . . . 7
- 3.2 Retry logika s validáciou Request ID . . . 8
  - 3.2.1 Strata paketov (timeout) . . . 8
  - 3.2.2 Oneskorené odpovede (starý Request ID) . . . 8
- 3.3 Validácia parsovaného paketu . . . 8
- 3.4 Filtrovanie numerických hodnôt pre Gauge metriky . . . 9

4 Návod na použitie **9**
- 4.1 Kompilácia . . . 9
- 4.2 Parametre programu . . . 9
- 4.3 Formát OID súboru . . . 10
- 4.4 Príklady použitia . . . 10
  - 4.4.1 Základné použitie . . . 10
  - 4.4.2 Použitie s retransmisiami . . . 10
  - 4.4.3 Verbose režim . . . 10
- 4.5 Ukončenie programu . . . 10
- 4.6 Návratové kódy . . . 10

5 Testovanie **10**
5.1 Unit testy . . . 11
5.1.1 SNMP PacketBuilder (15 testov) . . . 11
5.1.2 SNMP Parser (9 testov) . . . 11
5.1.3 OTELExporter (4 testy) . . . 11
5.2 Integračné testy . . . 11
5.2.1 Testy argumentov a buildu . . . 11
5.2.2 Testovanie s fake SNMP agentom a OTEL endpointom . . . 12
5.2.3 Testované scenáre . . . 12
5.3 Overenie funkčnosti . . . 13

6 Záver **14**

1 Uvedenie do problematiky

Zadaním projektu bolo vytvorenie nástroja na zber SNMP metrík typu *Gauge* zo sieťových zariadení a ich export do OpenTelemetry (OTEL) cez OTLP/HTTP, pričom podporuje periodické dotazovanie a export nameraných hodnôt pre ďalšiu analýzu.

1.1 Použité technológie a protokoly

1.1.1 SNMP (Simple Network Management Protocol)

SNMP je aplikačný protokol pre správu a monitorovanie sieťových zariadení. Program pracuje s verziou SNMPv2c, ktorá používa nešifrovaný community string a transport cez UDP. SNMP správy sú kódované v ASN.1 Basic Encoding Rules (BER) formáte [3].

1.1.2 ASN.1 BER kódovanie

ASN.1 (Abstract Syntax Notation One) je jazyk na popis dátových štruktúr. BER (Basic Encoding Rules) je binárny formát používaný SNMP protokolom. Každý element má štruktúru: Tag (typ), Length (dĺžka), Value (hodnota). Program implementuje kódovanie OID, INTEGER, OCTET STRING a SEQUENCE typov.

1.1.3 OpenTelemetry a OTLP/HTTP

OpenTelemetry je otvorený štandard pre dáta umožňujúce sledovanie a monitorovanie systémov (metriky, logy, ..). Program exportuje metriky pomocou protokolu OTLP/HTTP – odosiela HTTP POST request s JSON payloadom na endpoint. JSON obsahuje ResourceMetrics (napr. service.name, snmp.target), ScopeMetrics a Gauge metriky s dataPoints obsahujúcimi časové pečiatky v nanosekundách.

2 Návrh a implementácia

2.1 Architektúra programu

Program `snmp2otel` je modulárny:

- **SNMP_PacketBuilder** - tvorba SNMP paketov
- **SNMP_client** - UDP komunikácia
- **SNMP_parser** - parsovanie odpovedí
- **OTELExporter** - OTLP/HTTP export
- **main.cpp** - hlavná logika

2.2 SNMP Packet Builder

Modul `SNMP_PacketBuilder` zodpovedá za zostavenie validného SNMP GetRequest paketu podľa RFC 3416 [1]. Hlavné kroky sú:

2.2.1 Načítanie OID zo súboru

Funkcia `fileLoad()` načíta OID zo súboru v textovom formáte (jeden OID na riadok). Prázdne riadky a riadky začínajúce `#` sú ignorované. Každý riadok je validovaný, či obsahuje len číslice a bodky.

2.2.2 Konverzia OID na ASN.1 BER formát

Funkcia `encode_oids_to_ber()` implementuje OID kódovanie podľa ASN.1 BER špecifikácie [3]. Detailný popis algoritmu je uvedený v sekcii 3.1.

2.2.3 Zostavenie SNMP GetRequest PDU

Funkcia `make_getrequest_PDU()` zostavuje GetRequest PDU s nasledujúcou štruktúrou:

```
GetRequest-PDU ::= [0] IMPLICIT SEQUENCE {
    request-id      INTEGER (4 bytes),
    error-status    INTEGER (0),
    error-index     INTEGER (0),
    variable-bindings  VarBindList
}
```

Request ID sa inkrementuje pri každom dotaze, čo umožňuje rozlíšiť odpovede pri retransmisiách.

2.2.4 Finálne zostavenie SNMP paketu

Funkcia `make_message()` kombinuje GetRequest PDU s SNMP verziou (INTEGER, hodnota 1 pre SNMPv2c) a community stringom (OCTET STRING) do kompletného SNMP paketu obalený v vonkajšej SEQUENCE.

2.3 SNMP Client

Modul `SNMP_client` implementuje UDP komunikáciu so SNMP agentom.

2.3.1 Inicializácia socketu

Konštruktor vytvára UDP socket (`SOCK_DGRAM`), prekladá DNS meno/IP adresu cieľa pomocou `getaddrinfo()` a nastavuje receive timeout pomocou `setsockopt()` s parametrom `SO_RCVTIMEO`.

2.3.2 Odosielanie a príjem paketov

- `SNMP_send()` – odosiela SNMP paket pomocou `sendto()`
- `SNMP_receive()` – čaká na odpoveď pomocou `recvfrom()` s timeout-om. Pri timeout-e vracia prázdny vektor, čo signalizuje retransmisiu

2.4 SNMP Parser

Modul `SNMP_parser` parsuje SNMP GetResponse pakety.

2.4.1 Parsovanie štruktúry paketu

Funkcia `parse_response()` postupne parsuje:

1. Vonkajšiu SEQUENCE (celá SNMP správa)
2. SNMP verziu (INTEGER, očakávaná hodnota 1 pre SNMPv2c)
3. Community string (OCTET STRING)
4. GetResponse PDU (tag `0xA2`)

5. Request ID, error-status, error-index
6. VarBindList (SEQUENCE varbindov)

Pred začiatkom parsovania sa `offset` resetuje na 0. Pri akejkoľvek chybe (nesprávny tag, neočakávaná verzia) sa vracia prázdna `SNMPResponse` s `varbinds.size() == 0`.

2.4.2 Parsovanie OID

Funkcia `read_oid()` dekóduje OID z BER formátu:

- Prvý bajt sa rozdelí na prvé dva identifikátory: $X = \lfloor \text{bajt}/40 \rfloor$, $Y = \text{bajt} \bmod 40$
- Ďalšie bajty sa dekódujú z Base-128 formátu: ak MSB = 1, hodnota sa posúva o 7 bitov doľava a k nej sa pridáva ďalší bajt (opačný proces ako pri enkódovaní)

2.4.3 Pomocné parsovacie funkcie

Parser využíva množinu pomocných funkcií na parsovanie základných ASN.1 elementov:

- `read_sequence()` – overuje prítomnosť SEQUENCE tagu (0x30)
- `read_length()` – číta dĺžku elementu v BER formáte (podpora krátkej aj dlhej formy)
- `read_integer()` – parsuje INTEGER hodnoty (Request ID, error-status, error-index)
- `read_octet_string()` – parsuje OCTET STRING (community string, textové OID hodnoty)
- `read_object_id()` – overuje prítomnosť OID tagu (0x06)

2.4.4 Parsovanie hodnôt

Funkcia `read_value()` podporuje tieto ASN.1 typy:

- `0x02` – INTEGER (signed 32-bit)
- `0x04` – OCTET STRING (textový reťazec)
- `0x05` – NULL (žiadna hodnota)
- `0x41` – Counter32 (unsigned 32-bit)
- `0x42` – Gauge32 (unsigned 32-bit)
- `0x43` – TimeTicks (unsigned 32-bit, stotiny sekundy)

Všetky číselné hodnoty sa konvertujú na string pomocou `std::to_string()`.

2.5 OTEL Exporter

Modul `OTELExporter` exportuje metriky do OpenTelemetry Collector cez OTLP/HTTP.

2.5.1 Filtrovanie numerických hodnôt

Funkcia `build_metrics()` filtruje len číselné hodnoty pomocou `is_number()`, pretože podľa OpenTelemetry špecifikácie Gauge metriky môžu obsahovať len numerické hodnoty (`asInt` alebo `asDouble`) [5]. Textové hodnoty (OCTET STRING) sa ignorujú.

2.5.2 Zostavenie OTLP JSON

Funkcia `build_json()` generuje JSON v OTLP formáte podľa oficiálnej špecifikácie [7]. Štruktúra JSON vychádza z príkladov v OpenTelemetry Protocol repository [8]:

```
{
  "resourceMetrics": [{
    "resource": {
      "attributes": [
        {"key": "service.name", "value": {"stringValue": "snmp2otel"}},
        {"key": "snmp.target", "value": {"stringValue": "192.168.1.1"}}
      ]
    },
    "scopeMetrics": [{
      "scope": {"name": "snmp2otel", "version": "1.0.0"},
      "metrics": [{
        "name": "snmp.1.3.6.1.2.1.1.3.0",
        "gauge": {
          "dataPoints": [{
            "asInt": 745288,
            "timeUnixNano": "1763213856713103283",
            "attributes": [
              {"key": "oid", "value": {"stringValue": "1.3.6.1.2.1.1.3.0"}}
            ]
          }]
        }
      }]
    }]
  }]
}
```

Timestamp sa získava ako `std::chrono::system_clock::now()` konvertovaný na nanosekundy od Unix epoch.

2.5.3 HTTP POST pomocou cpp-httplib

Funkcia `send_json()` používa knižnicu `cpp-httplib` [9] na odoslanie HTTP POST requestu a knižnicu `nlohmann/json` [10] na vytvorenie JSON payload-u s hlavičkou `Content-Type: application/json`.

2.6 Hlavný program

2.6.1 Parsovanie argumentov

Funkcia `parseArgs()` používa `getopt()` na parsovanie argumentov príkazového riadku. Všetky povinné parametre (`-t`, `-o`, `-e`) sú validované.

2.6.2 Periodické dotazovanie

Hlavná slučka v `main()` implementuje:

1. Zostavenie SNMP GetRequest paketu
2. Odoslanie a čakanie na odpoveď s možnosťou retransmisie (až `config.retries + 1` pokusov)

3. Parsovanie odpovede s kontrolou Request ID (ignorovanie starých odpovedí)
4. Kontrola prázdnej response (`varbinds.empty()`) – pri zlom pakete sa pokus opakuje
5. Export do OTEL
6. Sleep do ďalšieho intervalu s možnosťou prerušenia cez `SIGINT`/`SIGTERM`

2.6.3 Retry logika

Pri timeout-e alebo chybe parsovania sa pokus opakuje až `config.retries`-krát. Ak ani po všetkých pokusoch nepríde validná odpoveď, vypíše sa chybové hlásenie a program pokračuje ďalším cyklom.

3 Zaujímavé časti implementácie

3.1 OID kódovanie v ASN.1 BER formáte

Jednou z častí implementácie bolo kódovanie OID do ASN.1 BER formátu. Algoritmus bol inšpirovaný dokumentáciou Microsoft [4] a funguje nasledovne:

1. Prvé dva identifikátory (X a Y) sa zakódujú do jedného bajtu ako: $40 \times X + Y$
2. Ďalšie identifikátory sa kódujú v Base-128 formáte s MSB flagom

Pre čísla väčšie ako 127 sa používa viacbajtové kódovanie, kde každý bajt obsahuje 7 bitov dát a MSB (8. bit) signalizuje pokračovanie. Napríklad: Číslo 311 sa zakóduje ako bajty `0x82 0x37`.

Implementovaný algoritmus v `encode_oids_to_ber()`:

```
// Encode first two components: (40 * X) + Y
uint32_t first_byte = oid_array[0]*40 + oid_array[1];
oid_in_ber.push_back(static_cast<uint8_t>(first_byte));

// Encode remaining sub-identifiers using Base-128
for (size_t i = 2; i < oid_array.size(); i++) {
    uint32_t value = oid_array[i];
    std::vector<uint8_t> bytes;

    // Split value into 7-bit chunks
    do {
        uint8_t byte = value & 0x7F;
        bytes.insert(bytes.begin(), byte);
        value >>= 7;
    } while (value > 0);

    // Set MSB=1 for all bytes except last (signals "more bytes follow")
    for (size_t j = 0; j < bytes.size() - 1; ++j) {
        bytes[j] |= 0x80;
    }

    oid_in_ber.insert(oid_in_ber.end(), bytes.begin(), bytes.end());
}
```

3.2 Retry logika s validáciou Request ID

Program implementuje retry logiku pre UDP komunikáciu, ktorá rieši dva problémy:

3.2.1 Strata paketov (timeout)

Pri UDP komunikácii môžu pakety zahynúť. Program používa konfigurovateľný počet retrát s timeoutom:

```
for (int attempt = 0; attempt <= config.retries; ++attempt) {
    client.SNMP_send(packet_to_send);
    received_packet = client.SNMP_receive();

    if (!received_packet.empty()) {
        break;  // Packet prijatý
    } else {
        std::cerr << "[WARN] Timeout, retry\n";
    }
}
```

3.2.2 Oneskorené odpovede (starý Request ID)

Pri viacerých pokusoch môže prísť odpoveď na starý request. Program to detekuje porovnaním Request ID:

```
if (parsed_response.request_id != current_req_id) {
    std::cerr << "[WARN] Old Request ID, retry\n";
    continue;  // Čakaj na správnu odpoveď
}
```

Request ID sa inkrementuje pri každom novom request-e, čo zabezpečuje, že program spracuje len aktuálnu odpoveď.

3.3 Validácia parsovaného paketu

Parser implementuje viacúrovňovú validáciu. Napríklad pred začiatkom parsovania sa overuje minimálna veľkosť a SEQUENCE tag:

```
SNMPResponse SNMP_parser::parse_response(data) {
    offset = 0;
    SNMPResponse response;

    if (data.size() < 20 || data[0] != 0x30) {
        std::cerr << "[ERROR] Invalid packet\n";
        return response;  // Prázdna response
    }
    // Pokračuj s parsovaním...
}
```

Pri akejkoľvek chybe počas parsovania sa okamžite vráti prázdna response s `varbinds.size() == 0`, čo main detekuje a spustí retry.

3.4 Filtrovanie numerických hodnôt pre Gauge metriky

Podľa OpenTelemetry špecifikácie môžu Gauge metriky obsahovať len numerické hodnoty (`asInt` alebo `asDouble`) [5]. Program implementuje filtrovanie pomocou funkcie `is_number()` inšpirovanej Stack Overflow [6]:

```
bool OTELExporter::is_number(const std::string& s) {
    return !s.empty() && std::find_if(s.begin(), s.end(),
        [](unsigned char c) { return !std::isdigit(c); }) == s.end();
}
```

Táto funkcia sa používa v `build_metrics()` na export len číselných SNMP hodnôt:

```
for (const auto& vb : response.varbinds) {
    if (is_number(vb.value)) {
        data.value = std::stoi(vb.value);
        metrics.push_back(data);
    }
}
```

Príklad: `sysUpTime` = "745288" → exportuje sa; `sysDescr` = "Linux router" → ignoruje sa.

4 Návod na použitie

4.1 Kompilácia

Program sa kompiluje pomocou `Makefile`:

```
make
```

Výsledkom je spustiteľný súbor `snmp2otel`.

4.2 Parametre programu

Program akceptuje nasledujúce parametre príkazového riadku:

- `-t <hostname>` – SNMP agent IP adresa alebo DNS meno (povinné)
- `-o <filepath>` – Cesta k súboru s OID (povinné)
- `-e <url>` – OTEL Collector endpoint URL (povinné)
- `-i <seconds>` – Interval dopytovania v sekundách (default: 10)
- `-r <count>` – Počet retransmisií pri timeout-e (default: 3)
- `-p <port>` – SNMP port (default: 161)
- `-c <string>` – SNMP community string (default: "public")
- `-v` – Verbose režim (debug výpisy)
- `-h` – Zobrazenie nápovedy

4.3 Formát OID súboru

Súbor s OID obsahuje jeden OID na riadok v desiatkovom formáte s bodkami:

```
# Komentár
1.3.6.1.2.1.1.3.0
1.3.6.1.2.1.1.7.0
```

Prázdne riadky a riadky začínajúce `#` sú ignorované.

4.4 Príklady použitia

4.4.1 Základné použitie

Export SNMP metrík na localhost každých 5 sekúnd:

```
./snmp2otel -t localhost -o oids.txt \
  -e http://localhost:4318/v1/metrics -i 5
```

4.4.2 Použitie s retransmisiami

Export s 5 pokusmi pri timeout-e a vlastným community stringom:

```
./snmp2otel -t 192.168.1.1 -o oids.txt \
  -e http://otel-collector:4318/v1/metrics \
  -i 10 -r 5 -c private
```

4.4.3 Verbose režim

Debug výpisy pre diagnostiku:

```
./snmp2otel -t localhost -o oids.txt \
  -e http://localhost:4318/v1/metrics -v
```

Výstup:

```
[DEBUG] OTEL endpoint: localhost:4318/v1/metrics
[DEBUG] Sending SNMP GetRequest...
[DEBUG] OTEL POST status: 200
```

4.5 Ukončenie programu

Program sa gracefully ukončí pri zachytení `SIGINT` (Ctrl+C) alebo `SIGTERM`. Aktuálny polling cyklus sa dokončí pred ukončením.

4.6 Návratové kódy

- `0` – Úspešné ukončenie
- `1` – Chybné parametre alebo zlyhanie počas behu

5 Testovanie

Program bol testovaný pomocou unit testov a integračných testov.

5.1 Unit testy

5.1.1 SNMP PacketBuilder (15 testov)

- Načítanie OID zo súboru (validácia, komentáre, prázdne riadky)
- OID kódovanie do BER formátu (základné čísla, veľké čísla >127, hranica 128)
- Parsovanie OID reťazcov na čísla
- Tvorba VarBind a VarBindList štruktúr
- Zostavenie GetRequest PDU (štruktúra, inkrementácia Request ID)
- Kódovanie community stringu
- Zostavenie kompletnej SNMP správy

5.1.2 SNMP Parser (9 testov)

- Parsovanie INTEGER hodnôt (malé aj veľké 4-bajtové čísla)
- Parsovanie TimeTicks (unsigned 32-bit)
- Parsovanie Gauge32 (unsigned 32-bit)
- Parsovanie Counter32 (unsigned 32-bit)
- Parsovanie OCTET STRING (textové hodnoty)
- Parsovanie NULL hodnôt
- Parsovanie viacerých packetov

5.1.3 OTELExporter (4 testy)

- Parsovanie URL (extrakcia hostname, portu, path)
- Konverzia OID na string formát
- Filtrovanie numerických hodnôt (ignorovanie OCTET STRING)

Výsledok: Všetkých 28 testov úspešne prešlo. Spustenie: `make test`

5.2 Integračné testy

5.2.1 Testy argumentov a buildu

Projekt obsahuje bash skript `tests/integration_tests.sh`, ktorý automaticky testuje základnú funkcionalitu:

1. **Build test** – Overenie kompilácie a vytvorenia spustiteľného súboru
2. **Help test** – Kontrola funkčnosti `-h` parametra
3. **Missing arguments test** – Validácia povinných argumentov
4. **Invalid file test** – Test s neexistujúcim OID súborom
5. **Invalid port test** – Test s neplatným portom (>65535)

5.2.2 Testovanie s fake SNMP agentom a OTEL endpointom

Boli vytvorené pomocné nástroje, ktoré boli generované pomocou Chatgpt čo zadanie povolovalo, slúžili na simuláciu SNMP agenta a OTEL Collector:

- `tests/fake_snmp.py` – Simulovaný SNMP agent (port 1611)
- `tests/fake_otel.py` – Simulovaný OTEL Collector (port 4318)

5.2.3 Testované scenáre

Test 1: Základný workflow (happy path)

Všetky komponenty fungujú správne, program úspešne načíta OID, odošle SNMP request, prijme response, parsuje hodnoty a exportuje do OTEL.

Výsledok: Program exportoval 2 metriky, fake OTEL prijal validný JSON. Výpis z fake OTEL serveru:


```
=== RECEIVED POST ===
Path: /v1/metrics
Content-Type: application/json
Content-Length: 456
{
  "resourceMetrics": [
    {
      "scopeMetrics": [
        {
          "metrics": [
            {
              "name": "snmp.1.3.6.1.2.1.1.3.0",
              "gauge": {
                "dataPoints": [
                  {"asInt": "123456"}
                ]
              }
            },
            {
              "name": "snmp.1.3.6.1.2.1.1.7.0",
              "gauge": {
                "dataPoints": [
                  {"asInt": "72"}
                ]
              }
            }
          ]
        }
      ]
    }
  ]
}
```

Test 2: Retry logika pri výpadku SNMP agenta

SNMP agent je vypnutý, program testuje retry mechanizmus s 2 pokusmi.

Výsledok: Program sa pokúsil 3×(1 + 2 retries), vypísal "[WARN] Timeout" pri každom pokuse, po 3. pokuse pokračoval ďalším cyklom.

Výpis z stderr:

```
Timeout: No SNMP response received.
[WARN] No response (timeout), retry 1/3
Timeout: No SNMP response received.
[WARN] No response (timeout), retry 2/3
Timeout: No SNMP response received.
[WARN] No response (timeout), retry 3/3
[ERROR] SNMP agent did not respond right after 2 retries
Timeout: No SNMP response received.
[WARN] No response (timeout), retry 1/3
```

Test 3: Výpadok OTEL endpointu

OTEL endpoint je vypnutý, testuje sa správanie pri HTTP chybe.

Výpis z stderr:

```
[ERROR] Failed to send JSON to OTEL endpoint: Could not establish connection (2)
```

Test 4: Zlý Request ID

Pre účely testu bol na fake SNMP agentovi “hardcoded” Request ID, ktoré neodpovedalo aktuálnemu dotazu. Program prijal viaceré odpovede so starým Request ID, ktoré boli správne detekované.

Výpis z stderr:

```
[WARN] Received response for old Request ID 999, expected 2
[WARN] Received response for old Request ID 999, expected 2
[WARN] Received response for old Request ID 999, expected 2
[ERROR] SNMP agent did not respond right after 2 retries
[WARN] Received response for old Request ID 999, expected 3
[WARN] Received response for old Request ID 999, expected 3
[WARN] Received response for old Request ID 999, expected 3
[ERROR] SNMP agent did not respond right after 2 retries
```

Test 5: Odpoveď na tretí pokus

Tento test simuluje situáciu, kedy SNMP agent neodpovie hneď, ale až po niekoľkých pokusoch. Fake SNMP server je nastavený tak, že odpoveď príde až na tretí pokus.

Výpis z stderr:

```
Timeout: No SNMP response received.
[WARN] No response (timeout), retry 1/3
Timeout: No SNMP response received.
[WARN] No response (timeout), retry 2/3
[DEBUG] OTEL POST status: 200
```

5.3 Overenie funkčnosti

Program bol úspešne testovaný so skutočným Net-SNMP agentom (`snmpd`) na Ubuntu 24.04. Porovnanie s `snmpget` utility ukázalo konzistentné výsledky. Program korektne exportoval číselné metriky a ignoroval textové hodnoty.

6 Záver

Program `snmp2otel` úspešne implementuje export SNMP Gauge metrík do OpenTelemetry podľa zadania. Podporuje periodické dotazovanie, retry logiku, validáciu paketov a graceful shutdown. Program bol otestovaný pomocou unit testov, integration testov aj so skutočným SNMP agentom.

Bibliografia

[1] Presuhn, R., et al., *Version 2 of the Protocol Operations for the Simple Network Management Protocol (SNMP)*, RFC 3416, December 2002. `https://www.rfc-editor.org/rfc/rfc3416.html`

[2] McCloghrie, K., et al., *Structure of Management Information Version 2 (SMIv2)*, RFC 2578, April 1999. `https://www.rfc-editor.org/rfc/rfc2578.html`

[3] Luca Deri, *ASN.1 and BER encoding*, `https://luca.ntop.org/Teaching/Appunti/asn1.html#BER`

[4] Microsoft, *About Object Identifier*, `https://learn.microsoft.com/en-us/windows/win32/seccertenroll/about-object-identifier`

[5] OpenTelemetry, *Metrics Data Model - Gauge*, `https://opentelemetry.io/docs/specs/otel/metrics/data-model/#gauge`

[6] Stack Overflow, *How to determine if a string is a number with C++*, `https://stackoverflow.com/questions/4654636/how-to-determine-if-a-string-is-a-number-with-c`

[7] OpenTelemetry, *OpenTelemetry Protocol Specification*, `https://github.com/open-telemetry/opentelemetry-proto`

[8] OpenTelemetry, *OpenTelemetry Protocol - Metrics Examples*, `https://github.com/open-telemetry/opentelemetry-proto/blob/main/examples/metrics.json`

[9] Hoshino, Y., *cpp-httplib: A C++ header-only HTTP/HTTPS server and client library*, `https://github.com/yhirose/cpp-httplib`

[10] Lohmann, N., *JSON for Modern C++*, `https://github.com/nlohmann/json`